Eksamen i Algoritmer og Datastrukturer (NDAA04010U)

(version med svar og forklaringer)

Københavns Universitet

2024-06-26

Instruktioner

Du har 4 timer til at besvare 22 opgaver, beskrevet nedenfor. Karakteren baseres på det samlede antal point (vægten af hver opgave er angivet) og på en helhedsbedømmelse. Opgavesættet har 19 nummererede sider. CLRS refererer til kursets lærebog, Introduction to Algorithms, 4th edition.

Skriftlige hjælpemidler samt brug af lokalt installeret software er tilladt. Det er *ikke* tilladt at tilgå internet eller at kommunikere med andre.

Multiple-choice opgaver besvares i ITX systemets multiple-choice modul. I hvert multiple-choice spørgsmål er det angivet hvorvidt der er præcis én korrekt svarmulighed eller om der skal angives en *mængde* af ét eller flere korrekte svar. Ved retning af multiple-choice opgaver gives der negative point for forkerte svar. Eksempler:

- Opgave X har 8 svarmuligheder, hvoraf 4 er korrekte. Hvert kryds ved en korrekt svarmulighed giver 1 point, og hvert kryds ved en forkert svarmulighed giver -1 point. Hvis du fx sætter kryds ved alle de korrekte svarmuligheder og én forkert svarmulighed får du 3 point. Hvis du ikke sætter nogen krydser får du 0 point, og sætter du krydser ved alle svarmuligheder får du også 0 point.

- Opgave Y har 5 svarmuligheder, hvoraf 1 er korrekt. Kryds ved den korrekte svarmulighed giver 4 point, og kryds ved en forkert svarmulighed giver -1 point. Sætter du fx kryds ved én svarmulighed, og den er forkert, giver opgaven totalt -1 point.

Svar på skriftlige opgaver afleveres som et Word dokument i ITX systemet. Kopiér gerne teksten fra den udleverede skabelon ind i besvarelsen. Der er mulighed for at tegne og skrive på tablet, og indsætte i dokumentet. Efter forsiden skal der laves *én side til hvert delspørgmål, i samme rækkefølge som i opgavesættet.* Angiv delspørgsmålets nummer og bogstav (fx 10.a) men kopiér ikke opgaveteksten ind i besvarelsen. Totalt skal Word dokumentet have 9 sider — hvis du ikke svarer på et delspørgsmål, så lav en tom side.

Multiple-choice

Dine svar skal indtastes i ITX-systemets multiple-choice modul.

1 MergeSort og sammenligninger (4%)

Betragt følgende input til MergeSort, implementeret som i CLRS sektion 2.3.

9	8	7	6

Hvor mange sammenligninger af to elementer laver MergeSort totalt på dette input? (Med andre ord, hvor mange gange udføres linje 13 i Merge proceduren?) Der er præcis ét korrekt svar, men det er tilladt at vælge flere svar hvis du er i tvivl.

1. 3 sammenligninger
2. 4 sammenligninger ✓
3. 5 sammenligninger
4. 6 sammenligninger
5. 7 sammenligninger

Svar: De rekursive kald på venstre og højre halvdel af input kræver hver 1 sammenligning. Den afsluttende Merge kræver yderligere 2 sammenligninger for at identificere de to mindste elementer, der begge kommer fra højre halvdel.

2 Rekursionsligninger (4%)

Hvilke af nedenstående rekursionsligninger, nummereret $i = 1, \ldots, 6$, har en løsning T_i hvor $T_i(n) = O(n \log n)$? Antag at $T_i(n) = 1$ for $n \leq 1$. Vælg ét eller flere korrekte svar.

Eksempler. Hvis $T_0(n) = \Theta(n)$ gælder også $T_0(n) = O(n \log n)$. Hvis $T_0(n) = \Theta(n^2)$ gælder *ikke* $T_0(n) = O(n \log n)$.

1. $T_1(n) = T_1(n/2) + \Theta(n)$ ✓
2. $T_2(n) = 2 \cdot T_2(n/2) + \Theta(n \log n)$
3. $T_3(n) = 2 \cdot T_3(n/2) + \Theta(n^2)$
4. $T_4(n) = 4 \cdot T_4(n/2) + \Theta(1)$
5. $T_5(n) = 4 \cdot T_5(n/2) + \Theta(n \log n)$
6. $T_6(n) = T_6(n-1) + \Theta(\log n)$ ✓

Svar: Rekursionsligningerne 1, 3, 4, 5 kan løses med master theorem og har løsninger henholdsvis $T_1(n) = \Theta(n)$, $T_2(n) = \Theta(n \lg^2 n)$, $T_3(n) = \Theta(n^2)$, $T_4(n) = \Theta(n^2)$, og $T_5(n) = \Theta(n^2)$. Rekursionsligning 2 kan løses med en generel udgave af master theorem eller ved hjælp af substitutionsmetoden. Rekursionsligning 6 kan løses som $T_6(n) = \Theta(n \log n)$ ved hjælp af substitutionsmetoden.

3 Del og hersk (4%)

En rekursiv algoritme A fungerer således når den køres på input x af størrelse n:

- Hvis $n = 1$, returneres svaret y i tid $\Theta(1)$.
- Ellers:
  - Der bruges tid $\Theta(n^2)$ på at beregne inputs x_1 og x_2, begge af størrelse $n/2$.
  - Algoritmen kaldes rekursivt på x_1 og x_2, hvor svarene y_1 og y_2 returneres.
  - Ud fra y_1 og y_2 beregnes y i tid $\Theta(n \log n)$, og y returneres.

For enkeltheds skyld kan du antage at n er en potens af 2, således at $n/2$ altid er et heltal. Hvilken af følgende rekursionsligninger beskriver bedst algoritmen A's køretid $T(n)$, kombineret med basistilfældet $T(1) = 1$? Der er præcis ét korrekt svar, men det er tilladt at vælge flere svar hvis du er i tvivl.

1. $T(n) = T(n/2) + 2 \cdot \Theta(1)$
2. $T(n) = T(n/2) + 2 \cdot \Theta(n \log n)$
3. $T(n) = T(n/2) + 2 \cdot \Theta(n^2)$
4. $T(n) = 2T(n/2) + \Theta(1)$
5. $T(n) = 2T(n/2) + \Theta(n \log n)$
6. $T(n) = 2T(n/2) + \Theta(n^2)$ ✓

Svar: Der er to rekursive kald på inputs af størrelse $n/2$. Ud over det bruges tid $\Theta(n^2) + \Theta(n \log n)$, som er $\Theta(n^2)$.

4 Løkkeinvarianter (4%)

Betragt følgende funktion, i pseudo-kode notationen fra CLRS, hvor input $A[1 : n]$ er et array af heltal og $n > 1$ er længden af A.

```
InTheLoop(A, n)
1  a = max{A[1], A[2]}
2  b = min{A[1], A[2]}
3  for i = 3 to n
4       if A[i] ≥ a
5            b = a
6            a = A[i]
7       else if A[i] ≥ b
8            b = A[i]
9  return b
```

Eksempel: For $n = 5$ og $A = \langle 6, 4, -2, 6, 7 \rangle$ returnerer InTheLoop værdien 6.

Hvilke udsagn er gyldige invarianter, der er sande umiddelbart efter udførelsen **if**-sætningen i linje 4–8 (*inden* variablen i bliver øget med 1)? Vælg ét eller flere korrekte svar.

1. $a \geq 0$
2. $b \geq 0$
3. $a \geq b$ ✓
4. $a > b$
5. $a = \max_{1 \leq j \leq i} A[j]$ ✓
6. $b = \min_{1 \leq j \leq i} A[j]$

Svar: Udsagn 1 og 2 gælder ikke efter 1 iteration hvis fx $A[1]$, $A[2]$, $A[3]$ er negative. Udsagn 3 og 5 gælder efter linje 2 og opretholdes i hver iteration uanset hvilke af betingelserne i **if** sætningen er sande. Udsagn 4 gælder ikke hvis fx alle indgange i A er identiske. Udsagn 6 gælder efter linje 2, men efter i iterationer er b lig med det *næststørste* element i $A[1 : i]$.

5 Algoritmeanalyse I (4%)

Betragt denne algoritme der tager et array $A[1 : n]$ af heltal som input og returnerer et array $B[1 : n]$.

```
MinAlgoritme(A, n)
1  Let B[1 : n] be a new array
2  k = 1
3  for i = 1 to n
4       if A[i] < A[k]
5            k = i
6       B[i] = A[k]
7  return B
```

Hvilke af følgende udsagn gælder altid om B, der returneres af MinAlgoritme(A, n), uanset indholdet af A? Vælg ét eller flere korrekte svar.

1. Sekvensen $B[1], \ldots, B[n]$ er ikke-aftagende.
2. Sekvensen $B[1], \ldots, B[n]$ er ikke-voksende.✓
3. B indeholder en delmængde af tallene fra A.✓
4. B indeholder det største tal fra A.
5. For hvert $i \in \{1, \ldots, n\}$ gælder $B[i] = \min_{1 \leq j \leq i} A[j]$.✓
6. For hvert $i \in \{1, \ldots, n\}$ gælder $B[i] = \min_{i \leq j \leq n} A[j]$.

Svar: Vi har $B[i] = \min_{1 \leq j \leq i} A[j]$, så B er ikke-voksende og tallene i B er en delmængde af tallene i A. B indeholder altid det mindste tal fra A, men ikke nødvendigvis det største (fx hvis det mindste element er $A[1]$).

6 Algoritmeanalyse II (4%)

Betragt denne algoritme der som argument tager et array $A[1 : n]$ med heltal. Algoritmen ændrer på indeholdet i A (call by reference), i stil med Quicksort.

```
SortHumor(A, n)
1  if n > 1
2       SortHumor(A, n − 1)
3       if A[n − 1] > A[n]
4            exchange A[n − 1] with A[n]
5       SortHumor(A, n − 1)
```

Hvilke af følgende udsagn gælder? Vælg ét eller flere korrekte svar.

1. SortHumor sorterer A i ikke-aftagende rækkefølge. ✓
2. SortHumor sorterer A i ikke-voksende rækkefølge.
3. Køretiden af SortHumor er $\Theta(n \log n)$.
4. Køretiden af SortHumor er $\Theta(n^2)$.
5. Køretiden af SortHumor er $\Theta(2^n)$.✓

Svar: Ved induktion i n ses at SortHumor sorterer A i ikke-faldende rækkefølge. Køretiden er givet ved rekursionsligningen $T(1) = 1$, $T(n) = 2T(n-1) + \Theta(1)$, der har løsningen $T(n) = \Theta(2^n)$.

7 Algoritmeanalyse III (4%)

Betragt denne algoritme, der tager et array $A[1:n]$ af heltal og to indekser i og j som input hvor $1 \leq i \leq j \leq n$.

```
MaksimalMystik(A, i, j)
1  if i == j
2        return A[i]
3  m = ⌊(i + j)/2⌋
4  return max{MaksimalMystik(A, i, m), MaksimalMystik(A, m + 1, j)}
```

Hvilke udsagn gælder, hvor n er længden af A? Vælg ét eller flere korrekte svar.

1. MaksimalMystik$(A, 1, n)$ returnerer altid tallet $A[1]$.
2. MaksimalMystik$(A, 1, n)$ returnerer det største tal i A. ✓
3. Køretiden for MaksimalMystik$(A, 1, n)$ er $\Theta(\log n)$.
4. Køretiden for MaksimalMystik$(A, 1, n)$ er $\Theta(n)$. ✓
5. Køretiden for MaksimalMystik$(A, 1, n)$ er $\Theta(n \log n)$.
6. MaksimalMystik følger del-og-hersk-paradigmet.✓
7. MaksimalMystik bruger dynamisk programmering.

Svar: MaksimalMystik følger del-og-hersk-paradigmet til at returnere det største tal i A, og køretiden kan beskrives som $T(n) = 2T(n/2)+\Theta(1)$, hvilket løses som $T(n) = \Theta(n)$. Koden bruger del-og-hersk, men ikke dynamisk programmering da der ikke er overlappende delproblemer.

8 Amortiseret analyse (4%)

Vi betragter træ repræsentationer af disjunkte mængder (eng. *disjoint sets*), som beskrevet i CLRS sektion 19.3, ved brug af heuristikken *union by rank*. Antag at der laves k_1 Make-Set operationer, k_2 Union operationer, og k_3 Find-Set operationer. Hvilke af nedenstående udsagn om den totale køretid for disse operationer er korrekte for alle værdier af k_1, k_2, k_3? Vælg ét eller flere korrekte svar.

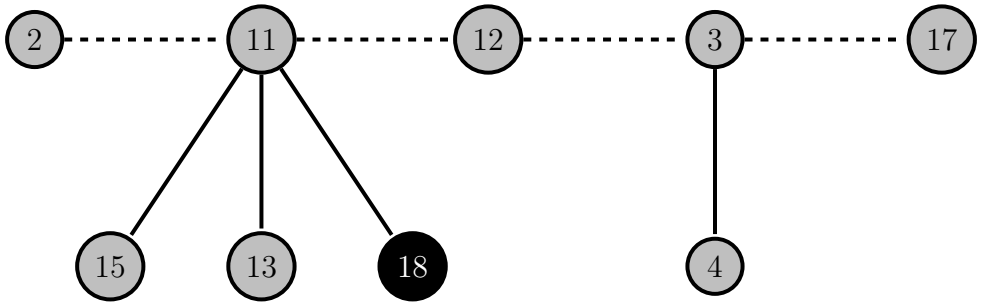
1. Tiden er $O(\log k_1 + \log k_2 + \log k_3)$
2. Tiden er $O(k_1 + \log k_2 + \log k_3)$
3. Tiden er $O(k_1 + k_2 + \log k_3)$

4. Tiden er $O(k_1 + k_2 + k_3)$
5. Tiden er $O((k_1 + k_2 + k_3) \log k_1)$ ✓
6. Tiden er $O((k_1 + k_2 + k_3) \log(k_1 + k_2 + k_3))$ ✓

Svar: Der laves $n = k_1$ singleton mængder og det totale antal operationer er $m = k_1 + k_2 + k_3$. Den samlede tid er $O(m \log n)$, dvs. $O((k_1 + k_2 + k_3) \log k_1)$. En mindre præcis grænse er $O(m \log m)$, dvs. $O((k_1 + k_2 + k_3) \log(k_1 + k_2 + k_3))$. De øvrige svarmuligheder er under disse tidsgrænser.

9 Fibonaccihobe (4%)

Denne opgave omhandler Fibonaccihobe (*eng., Fibonacci heaps*) som beskrevet i CLRS digitalt kapitel 19 (dvs. min-heap ordnede). Potentialfunktionen i analysen af Fibonaccihobe er $\Phi(H) = t(H) + 2\,m(H)$. Betragt nedenstående Fibonaccihob H. Som i CLRS figur 19.2.a illustreres rodlisten med stiplede linjer og markerede knuder tegnes med sort.



Hvilke af nedenstående operationer på H får værdien af potentialfunktionen til at *stige*? (Bemærk at CLRS ikke indeholder pseudokode for FIB-HEAP-MINIMUM, men kun en skriftlig beskrivelse.)

1. FIB-HEAP-MINIMUM(H)
2. FIB-HEAP-INSERT(H, 1) ✓
3. FIB-HEAP-INSERT(H, 20) ✓
4. FIB-HEAP-DELETE(H, 4)
5. FIB-HEAP-DELETE(H, 18)
6. FIB-HEAP-DECREASE-KEY(H, 11, 7)

Svar: FIB-HEAP-MINIMUM(H) ændrer ikke datastrukturen og derfor heller ikke potentialfunktionen. FIB-HEAP-INSERT operationerne øger rodlistens længde $t(H)$ med 1, og dermed også $\Phi(H)$. FIB-HEAP-DELETE operationer *kan* resultere i nye markerede knuder men i dette tilfælde laves ingen markeringer fordi kun knuder i rodlisten mister et barn. FIB-HEAP-DECREASE-KEY operationen resulterer ikke i nogen CUT operation, så potentialfunktionen ændres ikke.

10 Stavudskæring (4%)

Betragt algoritmen Bottom-Up-Cut-Rod beskrevet i CLRS sektion 14.1. Hvilket array r returnerer algoritmen på input $p = \langle 2, 3, 5, 7, 11, 12\rangle$, med $n = 6$? Der er præcis ét korrekt svar, men det er tilladt at vælge flere svar hvis du er i tvivl.

1. $r = \langle 0, 2, 4, 6, 8, 10, 12\rangle$
2. $r = \langle 0, 2, 4, 6, 8, 11, 12\rangle$
3. $r = \langle 0, 2, 4, 6, 8, 11, 13\rangle$ ✓
4. $r = \langle 0, 2, 4, 6, 9, 12, 13\rangle$
5. $r = \langle 0, 2, 4, 6, 9, 12, 14\rangle$
6. $r = \langle 0, 2, 4, 6, 9, 13, 14\rangle$

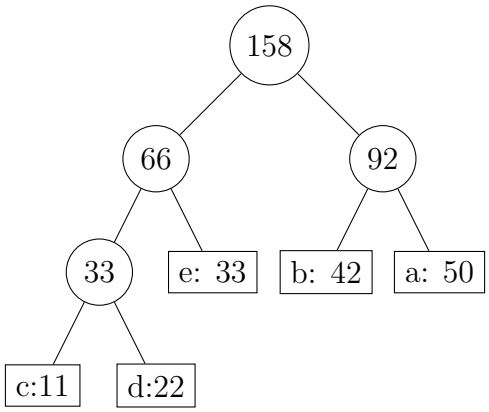
Svar: Inden første iteration har vi $r[0] = 0$. Efter 1 iteration har vi $r[1] = 2$ (maksimum fås ved at tage 1 stav af længde 1). Efter 2 iterationer har vi $r[2] = 4$ (maksimum fås ved at tage 2 stave af længde 1). Efter 3 iterationer har vi $r[3] = 6$ (maksimum fås ved at tage 3 stave af længde 1). Efter 4 iterationer har vi $r[4] = 8$ (maksimum fås ved at tage 4 stave af længde 1). Efter 5 iterationer har vi $r[5] = 11$ (maksimum fås ved at tage en stav af længde 5, andre løsninger giver en værdi på højst 10). Efter 6 iterationer har vi $r[6] = 13$ (maksimum fås ved at tage en stav af længde 5 og en af længde 1, andre løsninger giver en værdi på højst 12).

11 Huffman koder (4%)

Vi betragter algoritmen til konstruktion af Huffman koder i CLRS sektion 15.3 kørt på alfabetet $C = \{a, b, c, d, e\}$ med frekvenser $f_a = 50$, $f_b = 42$, $f_c = 11$, $f_d = 22$, $f_e = 33$. Hvor langt er det *korteste* kodeord (eng. *codeword*) i den beregnede Huffman kode? Der er præcis ét korrekt svar, men det er tilladt at vælge flere svar hvis du er i tvivl.

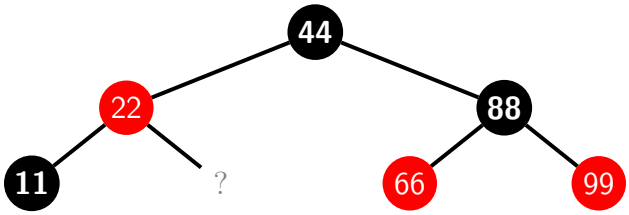
1. Længde 1
2. Længde 2 ✓
3. Længde 3
4. Længde 4

Svar: Algoritmen beregner dette træ (tegnet uden 0-1 labels på kanterne), der indeholder tre kodeord (for a,b,e) af længde 2 men ingen kodeord af længde 1:



12 Rød-sorte søgetræer I (4%)

Betragt et rød-sort binært søgetræ T (eng. *red-black binary search tree*), som beskrevet i CLRS kapitel 13. Betragt nedenstående træ, hvor alle blade er udeladt (som på CLRS figur 13.1.c) og én indre knude er uspecificeret:



Hvilke knuder er mulige på positionen markeret med ?, hvis træet skal overholde invarianterne for et rød-sort binært søgetræ? Der er præcis ét korrekt svar, men det er tilladt at vælge flere svar hvis du er i tvivl.

1. Sorte knuder med værdi mindst 11 og højst 22
2. Sorte knuder med værdi mindst 22 og højst 44 ✓
3. Sorte knuder med værdi mindst 11 og højst 66
4. Røde knuder med værdi mindst 11 og højst 22
5. Røde knuder med værdi mindst 22 og højst 44
6. Røde knuder med værdi mindst 11 og højst 66

Svar: Knuden skal være sort fordi dens forælder er rød, ellers ville en rød-sort invariant være overtrådt. Værdien skal være mellem 22 og 44 fordi træet er et søgetræ.

13 Rød-sorte søgetræer II (4%)

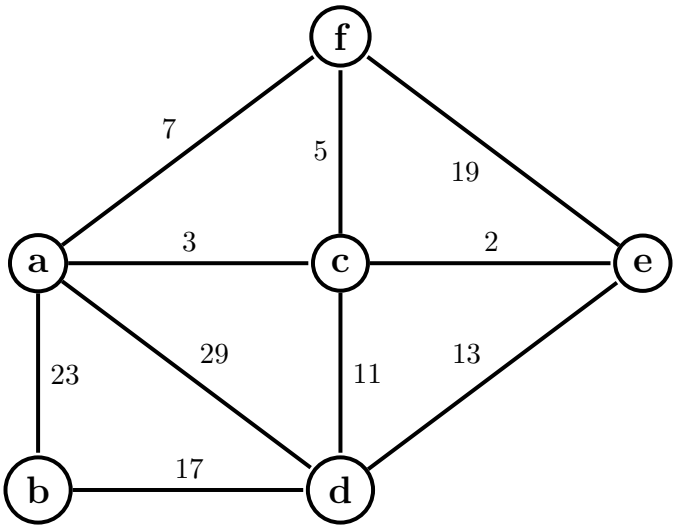
Betragt et rød-sort binært søgetræ T (eng. *red-black binary search tree*), som beskrevet i CLRS kapitel 13. Hvilke udsagn gælder for et hvilket som helst rød-sort binært søgetræ? Vælg ét eller flere korrekte svar.

1. Højden og den sorte højde er altid den samme for et blad (en ekstern knude). ✓
2. Den sorte højde stiger med 1 når man går ét skridt op i træet.
3. Antal røde knuder er altid mindre end antal sorte knuder (inkl. blade). ✓
4. Roden er altid sort. ✓
5. Der kan være både røde og sorte blade (eksterne knuder).
6. Når en knude indsættes er den rød, og den forbliver rød indtil en senere indsætning eller sletning.
7. Når en knude slettes, er det altid nødvendigt at omfarve og/eller rotere.

Svar: 1. Eksterne knuder har ingen børn og deres højde er derfor 0. Den sorte højde af en knude X er antallet af sorte knuder på vej ned igennem en vilkårlig sti fra X (X ikke medregnet). Hvis X er en ekstern knude er dens sorte højde er lig med 0. 2. Den sorte højde af en rød knude er lig med den sorte højde af dens far. 3. To børn af en rød knude er altid sorte. 4. Per definition af et rød-sort BST. 5. Ifølge definitionen af rød-sorte knuder er eksterne knuder altid sorte. 6. Simpelt modeksempel: Når en knude indsættes i et tomt rød-sort træ, får den rød farve. Dens farve ændres til sort fordi den er rod. 7. Når en knude skal slettes skal dens direkte efterfølger (eller direkte forgænger) findes. Hvis den er rød så skal den og knuden der skal slettes bytte plads. Nu kan den røde knude slettes og dets eventuelle højre undertræ skal hægtes på den slettedes knudes far som venstre undertræ.

14 Mindste udspændende træ (4%)

Betragt mindste udspændende træ (eng. *minimum spanning tree*) problemet på denne graf:



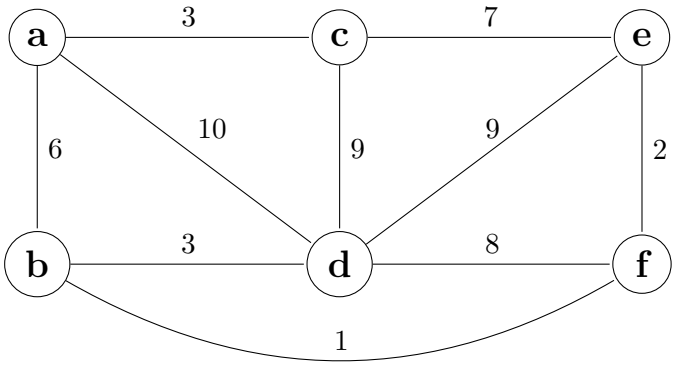
Hvilke kanter er med i det mindste udspændende træ (som er unikt)?

1. {a,b}
2. {a,c} ✓
3. {a,d}
4. {a,f}
5. {b,d} ✓
6. {c,d} ✓
7. {c,e} ✓
8. {c,f} ✓
9. {d,e}
10. {e,f}

Svar: Mindste udspændende træ kan findes ved at køre fx Kruskál's eller Prim's algoritme. Kruskal's algoritme tilføjer kanterne i vægtrækkefølge, dvs. starter med {c,e}, {a,c}, {c,f}, springer over {a,f} fordi det ville skabe en kreds, og tilføjer {c,d}, springer over {d,e} fordi det ville skabe en kreds, og afslutter med {b,d}.

15 Snit og sikre kanter (4%)

Betragt følgende uorienterede, vægtede graf med knudemængde $V = \{a, b, c, d, e, f\}$:



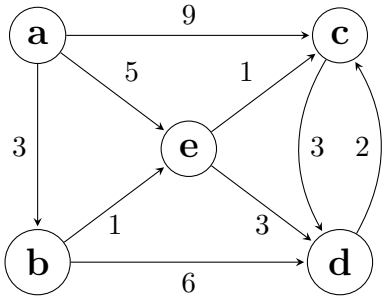
Hvilke kanter er lette kanter for snittet $(S, V - S)$ (eng. *light edges for the cut* $(S, V - S)$), hvor $S = \{a, b, e, f\}$? Vælg ét eller flere korrekte svar.

1. {a,b}
2. {a,c} ✓
3. {a,d}
4. {b,d} ✓
5. {b,f}
6. {c,d}
7. {c,e}
8. {d,e}
9. {d,f}
10. {e,f}

Svar: Den mindste vægt for kanter i snittet er 3, og de to snitkanter {a,c} og {b,d} har denne vægt. Kanterne {b,f} og {e,f} har lavere vægte men er ikke snitkanter.

16 Valg af algoritme (4%)

Betragt følgende orienterede, vægtede graf:



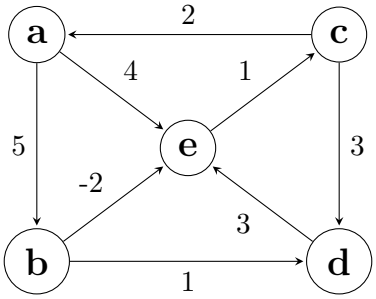
Hvilken af nedenstående algoritmer fra CLRS er det bedste valg til at beregne en korteste vej (eng. *single-source shortest-path*) fra startknude **a** til slutknude **d**? Der lægges vægt på korrekthed og god asymptotisk køretid for den valgte algoritme. Du kan antage at kantvægtene, i lighed med ovenstående graf, ligger mellem 1 og 9. Der er præcis ét korrekt svar, men det er tilladt at vælge flere svar hvis du er i tvivl.

1. Bellman-Ford algoritmen
2. Bredde-først søgning
3. Dijkstra's algoritme ✓
4. Kruskal's algoritme
5. Prim's algoritme

Svar: Bellman-Ford algoritmen og Dijkstra's algoritme beregner begge en korteste vej, men da grafen ikke har negative vægte er Dijkstra et bedre valg fordi den er asymptotisk hurtigere.

17 Korteste veje (4%)

Betragt korteste veje problemet (eng. *single-source shortest-paths problem*) med startknude **a** på følgende graf:



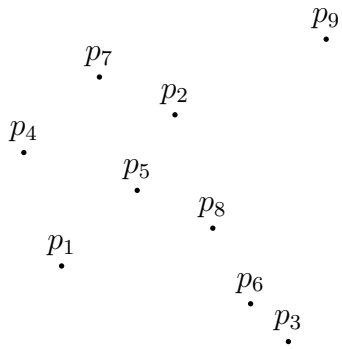
Hvilke kanter er med i korteste-vej træet (eng. *shortest-path tree*) med startknude **a**? Vælg ét eller flere korrekte svar.

1. (a,b) ✓
2. (a,e)
3. (b,d) ✓
4. (b,e) ✓
5. (c,a)
6. (c,d)
7. (d,e)
8. (e,c) ✓

Svar: Da der er negative vægte kan kanterne i træet findes ved at køre Bellman-Ford algoritmen, der giver det angivne output (som i dette tilfælde er unikt).

18 Beregningsgeometri (4%)

Det konvekse hylster (eng. *convex hull*) af en punktmængde kan beregnes med algoritmer fra CLRS digitalt kapitel 33. Betragt nedenstående punktmængde P:



Hvilke udsagn er korrekte? (De to første relaterer til punktmængden P, de øvrige handler om generelle punktmængder.) Vælg ét eller flere korrekte svar.

1. Det konvekse hylster for P har $h = 5$ hjørner. ✓
2. Punktet p_2 er ikke et hjørne i det konvekse hylster for P. ✓
3. Graham's scan er aldrig asymptotisk langsommere end Jarvis' march.
4. Jarvis' march tager tid $\Theta(n^2)$ på et værste input med n knuder. ✓
5. Graham's scan tager tid $\Theta(n^2)$ på et værste input med n knuder.

Svar: Det konvekse hylster har hjørnerne p_4, p_7, p_9, p_3, p_1, øvrige punkter er indeni det konvekse hylster. Graham's scan er langsommere end Jarvis' march når antal hjørner på det konvekse hylster er $h = o(\log n)$. Når $h = \Theta(n)$, fx hvis alle punkter ligger på en cirkel, tager Jarvis' march $\Theta(n^2)$ tid. Graham's scan tager $O(n \log n)$ tid.

Skriftlige opgaver

19 Længste fælles delsekvens (5%)

Vi betragter algoritmen til længste fælles delsekvens (eng. *longest common subsequence*) i CLRS sektion 14.4.

a) Tegn det array c som algoritmen udfylder på følgende input:

$$X = \langle N, E, O, N \rangle \text{ og } Y = \langle A, N, N, O, N, A \rangle.$$

Tilføj pile ($\uparrow$, $\nwarrow$, $\leftarrow$), der forklarer tallene i c, som i CLRS Figur 14.8 og *angiv den længste fælles delsekvens*. Skriv dit svar på side 2 i Word dokumentet.

Svar: For overskuelighedens skyld inkluderer vi de to strenge i kanten af c, ligesom i CLRS Figur 15.8.

		A	N	N	O	N	A
	0	0	0	0	0	0	0
N	0	$\uparrow$0	$\nwarrow$1	$\nwarrow$1	$\leftarrow$1	$\nwarrow$1	$\leftarrow$1
E	0	$\uparrow$0	$\uparrow$1	$\uparrow$1	$\uparrow$1	$\uparrow$1	$\uparrow$1
O	0	$\uparrow$0	$\uparrow$1	$\uparrow$1	$\nwarrow$2	$\leftarrow$2	$\leftarrow$2
N	0	$\uparrow$0	$\nwarrow$1	$\nwarrow$2	$\uparrow$2	$\nwarrow$3	$\leftarrow$3

(I nogle tilfælde er der et valg mellem $\uparrow$ og $\leftarrow$, det lægger vi ikke vægt på når der gives point.) Længste fælles delsekvens er NON.

20 Induktionsbevis for køretid (8%)

En algoritme tager tid $T(n) = n^2 + \sum_{i=1}^{n} \binom{i+1}{2}$ på et input af størrelse n, hvor $\binom{i+1}{2} = \frac{i(i+1)}{2}$. Vi ønsker at vise at køretiden overholder $T(n) \leq 2\,n^3$ for alle $n \geq 1$.

a) Vis at $T(1) = 2$. Skriv dit svar på side 3 i Word dokumentet.

Svar: $T(1) = 1^2 + \sum_{i=1}^{1} \binom{i+1}{2} = 2$.

b) Vis $T(n) \leq 2n^3$ ved induktion. Skriv dit svar på side 4 i Word dokumentet.

Svar: Fra del a) ved vi at $T(n) \leq 2\,n^3$ gælder i basistilfældet $n = 1$. I induktionsskridtet

antager vi at $T(n-1) \leq 2\,(n-1)^3$ (induktionshypotese). Vi har at

$$\begin{aligned} T(n) &= n^2 + \sum_{i=1}^{n} \tbinom{i+1}{2} \\ &= T(n-1) + (2n-1) + \tbinom{n+1}{2} \\ &\leq 2(n-1)^3 + (2n-1) + \tfrac{n(n+1)}{2} \\ &= (2\,n^3 - 6n^2 + 6n - 2) + (2n-1) + (n^2/2 + n/2) \\ &= 2\,n^3 - 5.5n^2 + 8.5n - 2.5 \\ &< 2\,n^3 \ . \end{aligned}$$

Den første ulighed bruger induktionshypotesen, den anden bruger at $-5.5n^2 + 8.5n - 2.5$ er negativt for $n \geq 2$. Dette færdiggør induktionsskridtet.

21 Køretid (9%)

Antag i det følgende at $A[1:n]$ er et array af n heltal og at k er et heltal. Vi betragter disse to funktioner:

PrefixSum(A, n, k)

```
1  s = 0
2  if k > n
3        t = n
4  else
5        t = k
6  for i = 1 to t
7        s = s + A[i]
8  return s
```

PowerSums(A, n)

```
1  k = 1
2  s = 0
3  while k < n
4        s = s + PrefixSum(A, n, k)
5        k = 2 · k
6  return s
```

a) Angiv en asymptotisk grænse på køretiden for PrefixSum med brug af store-Θ notation, som funktion af variablerne n og k. Argumentér kort for dit svar. Skriv dit svar på side 5 i Word dokumentet.

Svar: Køretiden er $\Theta(\min(n,k))$ fordi **for**-løkken laver t iterationer, hver iteration tager konstant tid, og **if** sætningen i linje 2-5 sikrer at $t = \min(n,k)$.

b) Angiv antal iterationer for **while** løkken i PowerSums med brug af store-Θ notation, som funktion af n. Argumentér kort for dit svar. Skriv dit svar på side 6 i Word dokumentet.

Svar: Løkken starter med $k = 1$, fordobler værdien af k i hver iteration, og terminerer når $k > n$. Per definition af totalslogaritmen af $n \geq 1$ er antal iterationer præcis $\lceil \lg n \rceil$. Dvs. antal iterationer er $\Theta(\lg n)$.

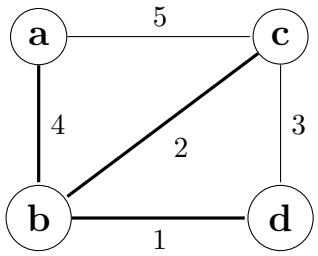
c) Angiv en øvre grænse for køretiden af PowerSums og udtryk den i store-Θ notation, som en simpel funktion af n. Argumentér kort for dit svar. Skriv dit svar på side 7 i Word dokumentet.

Svar: Ifølge spørgsmål a), og fordi $k < n$, er omkostningen ved hvert kald $\Theta(k)$. Iteration i sætter $k = 2^i$, så med brug af del b) bliver den samlede omkostning $\Theta(\sum_{i=0}^{\lceil \lg n \rceil} 2^i)$, som er $\Theta(n)$ da summen er en geometrisk række der domineres af det største led $2^{\lceil \lg n \rceil} < 2n$. Denne analyse er præcis da det sidste kald af PrefixSum har $k > n/2$, og dette kald alene tager tid $\Omega(n)$.

22 Minimax udspændende træ (6%)

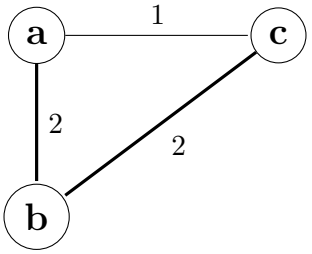
Betragt en vægtet, ikke-orienteret og sammenhængende graf $G = (V, E)$. Et udspændende træ i G hvor vægten af den tungeste kant er mindst muligt kaldes et *MiniMax Udspændende Træ* (MMUT).

Eksempel: I denne graf udgør de markerede kanter et MMUT. Den maksimale vægt i træet er 4, og der findes ikke noget udspændende træ med en mindre maksimal vægt.



a) Giv et eksempel på at et MMUT ikke altid er et mindste udspændende træ. Skriv dit svar på side 8 i Word dokumentet.

Svar: I denne graf findes et MMUT med maksimal vægt 2 og total vægt 4. Et mindste udspændende træ har vægt 3.



b) Bevis at et mindste udspændende træ altid er et MMUT. Skriv dit svar på side 9 i Word dokumentet.

Svar: Bevis ved modstrid. Lad T betegne et mindste udspændende træ (MUT) og antag at det ikke er et MMUT. Lad T' betegne et MMUT. Lad e betegne kanten med den største vægt i T og lad e' betegne kanten med den største vægt i T'. Da T ikke er et MMUT så må da gælde at $|e| > |e'|$ og e er ikke i T'. Når e tilføjes til T', dannes der en kreds. I denne kreds findes en kant der krydser snittet defineret ved sammenhængskomponenterne der opstår hvis e fjernes fra T, dvs. at e er ikke minimal i snittet, i modstrid med antagelsen.